



Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Compiladores - Prova 2 (25 pts)
Professor: Pedro Ramos

Estudante: _____

1. **Regras Semânticas (5pts).** Considere a gramática abaixo que gera strings de 0's e 1's:

$B \rightarrow B0$
$B \rightarrow B1$
$B \rightarrow 1 \quad \{B.valorDecimal = 1\}$

(a) Escreva as regras semânticas que computam o atributo `valorDecimal` do não terminal B na base decimal. Esse atributo computa o valor da string de 0's e 1's interpretada como um número inteiro positivo binário. Por exemplo, quando a gramática gera a string 111, o valor 7 deve ser computado na raiz da AST. Obs: a regra semântica da produção final já está implementada. (4pts)

(b) O atributo `valorDecimal` é herdado ou sintetizado? Justifique. (1pt)

2. **Tradução Dirigida por Sintaxe (7pts).**

Na gramática abaixo, o não-terminal T gera um tipo básico ou um tipo arranjo (*array*). O não-terminal B gera um dos dois tipos básicos `int` e `float`. T gera um tipo básico quando T deriva BC e C deriva λ . Caso contrário, C deriva um array composto de uma sequência de inteiros, cada inteiro ao redor de colchetes:

PRODUÇÃO	REGRAS SEMÂNTICAS
$T \rightarrow BC$	$T.tipo = C.tipo$ $C.basico = B.tipo$
$B \rightarrow \text{int}$	$B.tipo = integer$
$B \rightarrow \text{float}$	$B.tipo = float$
$C \rightarrow [\text{num}] C_1$	$C.tipo = array(\text{num.valor}, C_1.tipo)$ $C_1.basico = C.basico$
$C \rightarrow \lambda$	$C.tipo = C.basico$

Os não-terminais B e T possuem um atributo sintetizado `tipo`. O não terminal C possui dois atributos: o atributo herdado `basico` e o atributo sintetizado `tipo`. O atributo herdado `basico` passa um tipo básico para baixo na árvore, e o sintetizado `tipo` acumula o resultado. Dito isso, para a seguinte string de entrada:

`int [2] [3]`

- (a) Desenhe a árvore de sintaxe abstrata da string. (2pts)
- (b) Mostre o resultado da tradução dirigida por sintaxe para a AST obtida. Isto é, mostre a sequência correta de regras semânticas aplicadas às produções durante a travessia na AST. (5pts)

3. Despacho Dinâmico (8pts).

Considere o programa abaixo escrito em C++:

```
1 class Pessoa {
2 public:
3     virtual void seApresentar();
4 };
5
6 class Professor : public Pessoa {
7 public:
8     void seApresentar() override { std::cout << "Oi"; }
9     void lancarNota(int nota) {
10         std::cout << "Nota: " << nota;
11     }
12 };
13
14 int main() {
15     Pessoa* p = new Professor();
16     p->seApresentar();
17     p->lancarNota(25);
18     return 0;
19 }
```

- (a) O programa compila sem erros de tipagem estática? Justifique. (2pts)
- (b) Caso sua resposta para (a) tenha sido **não**, proponha uma solução para a compilação desse programa. Qual mecanismo permitido pela linguagem pode ser utilizado? (2pts)
- (c) Explique como o compilador gera código assembly para a chamada (despacho) da função **seApresentar**. Como o compilador encontra o endereço da implementação da linha 8? (3pts)
- (d) Indique uma linguagem de programação que não impede a compilação de programas como este. A linguagem indicada é mais **sound** que C++? Justifique. (1pt)
4. **Geração de Código Assembly (5pts).** Gere código MIPS para a declaração de função abaixo, que contém um operador ternário, e sua execução na linha 6. O código gerado deve manter a **semântica** da função original. Escreva todo o código da função invocada (*callee*) e invocadora (*caller*).

```
1 function f(a, b) {
2     i := a == b ? 5 : 0
3     return i + 1
4 }
5
6 f(10, 15)
```

Registadores MIPS de uso comum:

\$zero (0), \$v0-\$v1 (2-3: Retorno), \$a0-\$a3 (4-7: Argumentos), \$t0-\$t7 (8-15: Temporários), \$s0-\$s7 (16-23: Salvos), \$t8-\$t9 (24-25: Temporários), \$sp (29: Stack Pointer), \$ra (31: Endereço de Retorno).

Instrução	Tipo	Descrição	Operação RTL
<i>Aritméticas e Lógicas</i>			
add \$d, \$s, \$t	R	Soma (com overflow)	$d = s + t$
addu \$d, \$s, \$t	R	Soma (sem overflow)	$d = s + t$
sub \$d, \$s, \$t	R	Subtração (com overflow)	$d = s - t$
subu \$d, \$s, \$t	R	Subtração (sem overflow)	$d = s - t$
addi \$t, \$r, imm	I	Soma Imediata (com overflow)	$t = r + \text{imm}$
addiu \$t, \$r, imm	I	Soma Imediata (sem overflow)	$t = r + \text{imm}$
mult \$s, \$t	R	Multiplicação (32×32 bits)	Hi, Lo = $s * t$
div \$s, \$t	R	Divisão	Lo = s / t ; Hi = $s \% t$
and \$d, \$s, \$t	R	E Lógico (AND)	$d = s \& t$
or \$d, \$s, \$t	R	OU Lógico (OR)	$d = s t$
xor \$d, \$s, \$t	R	OU Exclusivo (XOR)	$d = s \wedge t$
nor \$d, \$s, \$t	R	NÃO OU Lógico (NOR)	$d = \sim(s t)$
andi \$t, \$r, imm	I	E Lógico Imediato	$t = r \& \text{imm}$
ori \$t, \$r, imm	I	OU Lógico Imediato	$t = r \text{imm}$
sll \$d, \$t, shamt	R	Deslocamento Lógico à Esquerda	$d = t \ll \text{shamt}$
srl \$d, \$t, shamt	R	Deslocamento Lógico à Direita	$d = t \gg \text{shamt}$
sra \$d, \$t, shamt	R	Deslocamento Aritmético à Direita	$d = t \gg \text{Arit. shamt}$
<i>Acesso à Memória</i>			
lw \$t, offset(\$r)	I	Carrega Word (32 bits) da memória	$t = \text{Mem}[r + \text{offset}]$
lh \$t, offset(\$r)	I	Carrega Halfword (com sinal)	$t = \text{Mem}[r + \text{offset}]$
lb \$t, offset(\$r)	I	Carrega Byte (com sinal)	$t = \text{Mem}[r + \text{offset}]$
sw \$t, offset(\$r)	I	Armazena Word na memória	$\text{Mem}[r + \text{offset}] = t$
sh \$t, offset(\$r)	I	Armazena Halfword na memória	$\text{Mem}[r + \text{offset}] = t$
sb \$t, offset(\$r)	I	Armazena Byte na memória	$\text{Mem}[r + \text{offset}] = t$
lui \$t, imm	I	Carrega Constante em Upper 16 bits	$t = \text{imm} \ll 16$
<i>Controle de Fluxo e Saltos</i>			
beq \$r, \$t, label	I	Desvia se Igual	if ($r == t$) goto label
bne \$r, \$t, label	I	Desvia se Diferente	if ($r != t$) goto label
j label	J	Salto Incondicional	goto label
jal label	J	Salto e Vínculo (Chamada de Função)	$\$ra = PC + 4$; goto label
jr \$r	R	Salto via Registrador (Retorno)	goto r
<i>Comparação e Especiais</i>			
slt \$d, \$s, \$t	R	Define se Menor que (com sinal)	$d = (s < t) ? 1 : 0$
slti \$t, \$r, imm	I	Define se Menor que Imediato	$t = (r < \text{imm}) ? 1 : 0$
mfhi \$d	R	Move do registrador Hi	$d = \text{Hi}$
mflo \$d	R	Move do registrador Lo	$d = \text{Lo}$
syscall	R	Chamada de Sistema	Depende de \$v0